Universidad Don Bosco



Asignatura: Datawarehouse y Minería de Datos

Anteproyecto - Fase 1

Data Warehouse y modelo multidimensional de NYC311

Integrantes

Nombre	Carnet
Lemus Mejía, Fernando José	LM212528
Bonilla Saz, Gerardo Rafael	BS242634
Chávez Castillo, Edson Enrique	CC242635
López Ortiz, César Giovanni	LO232756
Galdámez Soto, Juan José	GS242646

Soyapango, 10 de agosto de 2026

Contenido

1. Introducción 3
2. Objetivos 3
3. Caso de estudio y contexto organizacional 3
4. Proceso de negocio analizado 4
5. Problema de negocio y justificación del Data Warehouse 4
6. Fuentes de datos 5
7. Modelo multidimensional del Data Warehouse 7
8. Script de creación (DDL) 9
9. Proceso ETL 10
10. Conclusiones de la fase 15
11. Referencias 15
Anexo A. Script DDL completo 16

1. Introducción

NYC311 concentra las solicitudes de servicios municipales no urgentes de la Ciudad de Nueva York y registra, para cada caso, información sobre el problema reportado, la agencia responsable, la ubicación, el estado y las fechas asociadas a su atención. Este volumen de registros permite estudiar el comportamiento de la demanda ciudadana, siempre que la información se prepare para un análisis histórico y consistente.

El presente anteproyecto plantea la construcción de un Data Warehouse orientado a ese análisis. La propuesta integra la información oficial de NYC311, aplica un proceso ETL para extraerla, tiparla y validar su calidad, y finalmente la organiza en un modelo multidimensional que permite consultar las solicitudes por tiempo, agencia, problema, geografía, estado y canal de ingreso.

La primera implementación trabaja con las solicitudes registradas durante julio de 2026. El período se utiliza como una ventana controlada para validar el flujo completo -desde la API hasta la tabla de hechos- sin limitar el diseño a un único mes; la estructura puede reutilizarse posteriormente con otros rangos de fechas.

El resultado de esta fase constituye la base técnica de las etapas posteriores del proyecto, en las que el Data Warehouse será utilizado para minería de datos y visualización de información.

2. Objetivos

2.1 Objetivo general

Facilitar el análisis histórico y multidimensional de las solicitudes ciudadanas de NYC311 mediante un Data Warehouse que permita identificar tendencias en la demanda, tiempos de resolución, recurrencia de problemas y distribución geográfica.

2.2 Objetivos específicos

1. Identificar y analizar la fuente de datos oficial utilizada para la gestión de solicitudes ciudadanas de NYC311.
2. Definir el proceso de negocio, la granularidad y el nivel de detalle requerido para el análisis de las solicitudes.
3. Diseñar un modelo multidimensional mediante un esquema estrella que permita analizar las solicitudes desde diferentes perspectivas.
4. Definir la tabla de hechos, las métricas y las dimensiones necesarias para representar el proceso de gestión de solicitudes ciudadanas.
5. Implementar un proceso ETL que permita extraer, limpiar, transformar y cargar los datos desde la fuente original hacia el Data Warehouse.
6. Aplicar reglas de validación y control de calidad que permitan identificar y separar registros inconsistentes antes de su incorporación al modelo analítico.
7. Construir una estructura de datos que sirva como base para posteriores consultas analíticas, procesos de Minería de Datos y herramientas de Inteligencia de Negocios.

3. Caso de estudio y contexto organizacional

3.1 NYC311

NYC311 es el punto de acceso de la Ciudad de Nueva York para obtener información y gestionar servicios municipales que no corresponden a situaciones de emergencia. A través de la plataforma, los ciudadanos pueden reportar problemas o solicitar servicios relacionados con ruido, estacionamiento ilegal, basura, calefacción, mantenimiento de viviendas, baches, transporte y otros asuntos municipales.

La plataforma centraliza las solicitudes y las canaliza hacia las agencias municipales responsables de su atención. Cada registro conserva información que permite reconocer qué ocurrió, quién debe atenderlo, dónde se reportó y en qué momento del ciclo de atención se encuentra.

3.2 Funcionamiento general del servicio

Cuando un ciudadano registra una solicitud, NYC311 genera un identificador único y clasifica el caso de acuerdo con el tipo de problema reportado. Después, la solicitud es asignada a la agencia municipal correspondiente.

Durante su ciclo de vida, la solicitud puede permanecer abierta, asignada o en proceso mientras la agencia realiza las acciones necesarias. Una vez finalizada la atención, se registra el cierre y, cuando está disponible, una descripción de la resolución aplicada.

4. Proceso de negocio analizado

4.1 Gestión y atención de solicitudes

El proceso de negocio seleccionado es la gestión y atención de solicitudes ciudadanas no urgentes registradas mediante NYC311. El análisis sigue cada solicitud desde su registro hasta su atención o cierre, considerando el problema reportado, la agencia responsable, la ubicación, el estado y las fechas asociadas al proceso.

4.2 Etapas del proceso

El proceso se resume en las siguientes etapas:

- **Recepción:** el ciudadano realiza una solicitud o reporta un problema mediante alguno de los canales disponibles.
- **Registro:** NYC311 genera un identificador único para la solicitud.
- **Clasificación:** la solicitud se asocia con un tipo de problema y una descripción.
- **Asignación:** el caso se envía a la agencia municipal responsable.
- **Atención:** la agencia realiza las acciones necesarias para atender el caso.
- **Seguimiento:** la solicitud permanece registrada mientras se encuentra pendiente o en proceso.
- **Cierre:** una vez finalizada la atención, se registra la fecha de cierre y, cuando existe, la descripción de resolución.

4.3 Evento de negocio y granularidad

El evento analizado es el registro de una solicitud ciudadana en NYC311. Cada solicitud constituye una ocurrencia individual del proceso y define la granularidad del modelo: una fila en FactSolicitud por cada solicitud registrada.

5. Problema de negocio y justificación del Data Warehouse

5.1 Problemática

NYC311 reúne solicitudes asociadas a múltiples problemas, agencias y ubicaciones. El registro operativo es adecuado para consultar casos individuales, pero no está organizado para comparar de forma directa el comportamiento histórico de la demanda, los tiempos de atención o la concentración territorial de los problemas.

Para una revisión gerencial, consultar solicitudes de manera aislada obliga a repetir filtros, conversiones y reglas de calidad cada vez que se desea comparar períodos o grupos. Esto dificulta mantener indicadores consistentes y hace más costoso responder preguntas que combinan tiempo, agencia, problema y ubicación.

5.2 Necesidad analítica

El Data Warehouse se propone como una capa analítica separada del registro operativo. Su función es consolidar la información ya validada y mantener una estructura estable para comparar solicitudes desde las mismas dimensiones y con las mismas reglas de cálculo.

Esta estructura permite pasar de la consulta de tickets individuales a la observación de tendencias: dónde se concentra la demanda, qué problemas se repiten, qué agencias reciben mayor carga y cuánto tarda la resolución cuando las fechas disponibles son válidas.

5.3 Indicadores de análisis

Indicador	Cálculo	Lectura de negocio
Volumen de solicitudes	Conteo de registros de FactSolicitud	Demanda recibida por período, agencia, problema o zona.
Tiempo de resolución	minutos_resolucion	Duración de una solicitud cerrada con fechas válidas.
Tiempo promedio de resolución	Promedio de minutos_resolucion	Comparación de tiempos entre agencias, problemas, zonas y períodos.
Recurrencia de problemas	Conteo por complaint_type / descriptor	Problemas que se repiten con mayor frecuencia.
Distribución geográfica	Conteo por borough, ZIP o división territorial	Zonas donde se concentra la demanda.

5.4 Preguntas gerenciales

- ¿Qué tipos de solicitudes se presentan con mayor frecuencia?
- ¿Qué agencias reciben mayor cantidad de solicitudes?
- ¿Qué zonas geográficas concentran determinados problemas?
- ¿Qué tipos de solicitudes presentan mayores tiempos de resolución?
- ¿Qué solicitudes permanecen abiertas durante más tiempo?
- ¿Qué problemas presentan mayor recurrencia?
- ¿Cómo cambia la demanda de solicitudes a través del tiempo?
- ¿Qué combinaciones de agencia, problema y ubicación presentan mayores niveles de demora?

5.5 Alcance del proyecto

La primera fase utiliza las solicitudes registradas durante julio de 2026. Este período funciona como conjunto inicial para desarrollar y validar la extracción, las reglas de calidad, la carga dimensional y la tabla de hechos. La estructura del Data Warehouse no depende de ese mes y puede ejecutarse con otros rangos de fechas mediante los parámetros del proceso ETL.

6. Fuentes de datos

6.1 NYC Open Data

La fuente principal es NYC Open Data, plataforma oficial de datos abiertos de la Ciudad de Nueva York. El proyecto utiliza la información publicada para las solicitudes de servicio de NYC311.

6.2 Dataset erm2-nwe9

El conjunto seleccionado es “311 Service Requests from 2020 to Present”, identificado por el código erm2-nwe9. El dataset contiene, entre otros datos, fechas de creación y cierre, agencia responsable, tipo de problema, ubicación, estado, canal de registro y descripción de resolución.

6.3 Campos seleccionados

Grupo	Campos principales	Uso en el proceso
Identificación	unique_key	Identificador de la solicitud.
Fechas	created_date, closed_date, due_date	Análisis temporal y reglas de calidad.
Agencia	agency, agency_name	Responsable de la atención.
Problema	complaint_type, descriptor, descriptor_2, location_type	Clasificación del incidente.

Grupo	Campos principales	Uso en el proceso
Ubicación	borough, incident_zip, city	Análisis territorial.
Divisiones territoriales	community_board, council_district, police_precinct	Detalle geográfico adicional.
Dirección y coordenadas	incident_address, address_type, latitude, longitude	Localización del incidente.
Estado	status	Situación de la solicitud.
Resolución	resolution_description	Información registrada al atender el caso.
Canal	open_data_channel_type	Medio de ingreso de la solicitud.

6.4 Método de extracción mediante API SODA

La extracción se realiza mediante la API SODA de NYC Open Data. El paquete recibe un rango definido por StartFrom y EndAt, solicita los campos utilizados por el proyecto y procesa la respuesta en lotes. En la implementación actual, el Script Component pagina la consulta con lotes de 10,000 registros y utiliza offset para continuar hasta completar el período solicitado.

El componente contempla reintentos ante fallos temporales de la consulta. Los registros obtenidos se almacenan primero en RAW antes de aplicar tipado y validaciones, de forma que la carga analítica no dependa directamente de la respuesta de la API.

7. Modelo multidimensional del Data Warehouse

A partir del proceso de negocio se diseñó un esquema estrella con FactSolicitud como tabla central. El modelo conserva una fila por solicitud y relaciona el hecho con dimensiones de tiempo, agencia, problema, geografía, estado y canal. La dimensión de fecha se reutiliza para representar tanto la creación como el cierre de la solicitud.

7.1 Diagrama entidad-relación (esquema estrella)

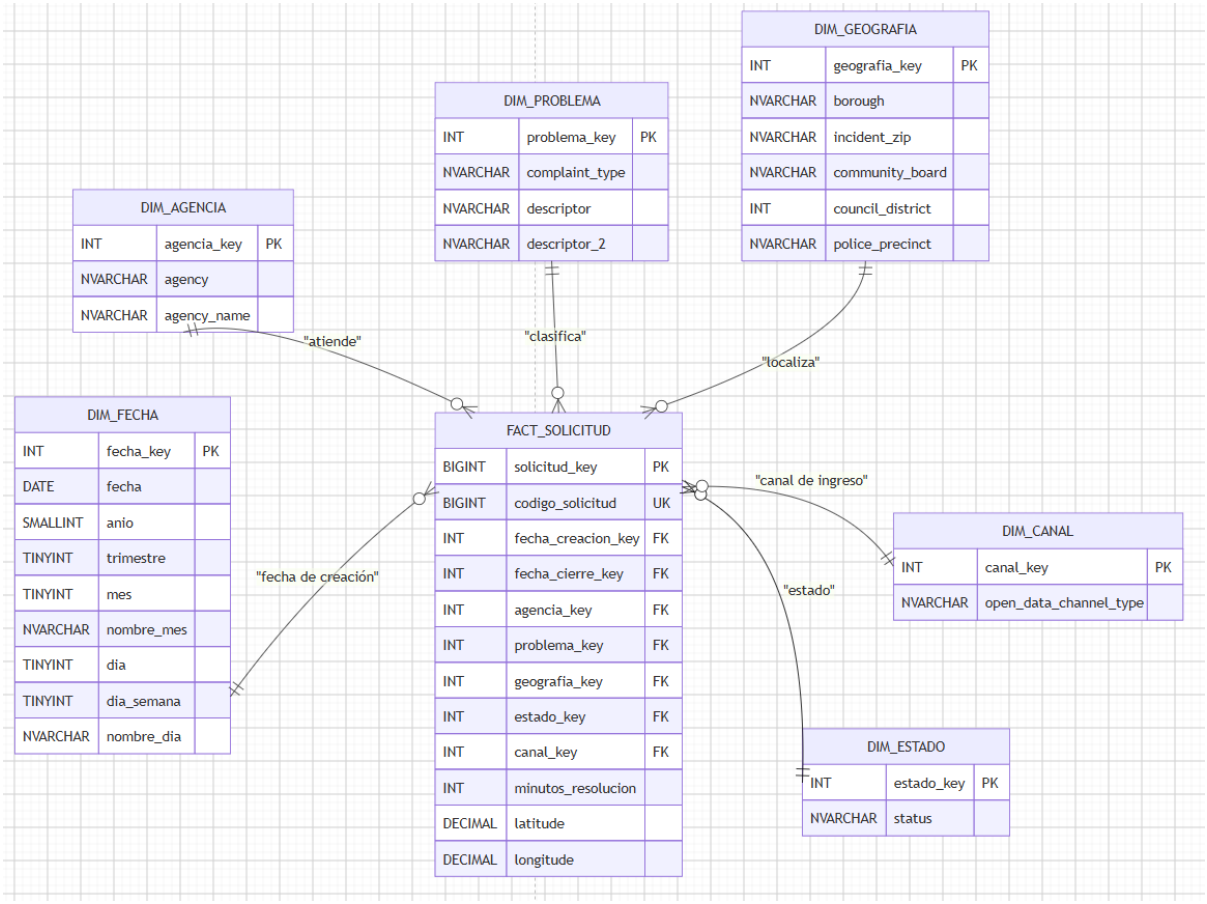


Figura 1. Modelo entidad-relación del esquema estrella de DW_NYC311.

7.2 Tabla de hechos: FactSolicitud

FactSolicitud representa el proceso de gestión de solicitudes de NYC311. Su granularidad es una fila por solicitud registrada.

Campo	Tipo	Descripción
solicitud_key	Llave subrogada	Identificador interno de la tabla de hechos.
codigo_solicitud	Identificador de negocio	Código original unique_key de NYC311.
fecha_creacion_key	Llave foránea	Fecha en que se registró la solicitud.
fecha_cierre_key	Llave foránea	Fecha en que se cerró la solicitud; puede ser nula.
agencia_key	Llave foránea	Agencia responsable.
problema_key	Llave foránea	Tipo de problema reportado.
geografia_key	Llave foránea	Ubicación de la solicitud.
estado_key	Llave foránea	Estado de la solicitud.
canal_key	Llave foránea	Canal de ingreso.
minutos_resolucion	Medida	Minutos transcurridos entre creación y cierre cuando las fechas son válidas.
latitude	Atributo geográfico	Latitud del incidente.
longitude	Atributo geográfico	Longitud del incidente.

7.3 Métricas

Métrica	Obtención	Utilidad analítica
Cantidad de solicitudes	Conteo de FactSolicitud	Volumen de demanda por las dimensiones del modelo.

Métrica	Obtención	Utilidad analítica
Tiempo de resolución	minutos_resolucion	Tiempo de atención de una solicitud cerrada con fechas válidas.
Tiempo promedio de resolución	Promedio de minutos_resolucion	Comparación de tiempos entre agencias, problemas, zonas y períodos.

7.4 Tablas de dimensión

Dimensión	Atributos principales	Propósito analítico
DimFecha	fecha, anio, trimestre, mes, nombre_mes, dia, dia_semana, nombre_dia	Analizar creación y cierre a través del tiempo.
DimAgencia	agency, agency_name	Comparar solicitudes y tiempos entre agencias.
DimProblema	complaint_type, descriptor, descriptor_2	Clasificar el problema reportado.
DimGeografia	borough, incident_zip, community_board, council_district, police_precinct	Analizar distribución territorial.
DimEstado	status	Analizar las solicitudes según su estado.
DimCanal	open_data_channel_type	Comparar medios de ingreso.

Las dimensiones utilizan llaves internas para desacoplar el modelo analítico de los identificadores de la fuente. DimFecha actúa como una dimensión de rol: fecha_creacion_key y fecha_cierre_key apuntan a la misma tabla con significados distintos.

8. Script de creación (DDL)

El modelo multidimensional fue implementado en SQL Server mediante instrucciones DDL para crear la base DW_NYC311, el esquema dw, las seis dimensiones, FactSolicitud y sus restricciones de integridad. El script completo se incluye en el Anexo A para conservar la trazabilidad de la implementación sin interrumpir la lectura del modelo.

8.1 Estructuras creadas

Objeto	Función
dw.DimFecha	Dimensión temporal para creación y cierre.
dw.DimAgencia	Agencia responsable.
dw.DimProblema	Clasificación del problema.
dw.DimGeografia	Perspectiva territorial.
dw.DimEstado	Estado de atención.
dw.DimCanal	Canal de registro.
dw.FactSolicitud	Hecho con una fila por solicitud y la medida minutos_resolucion.

8.2 Fragmento representativo

```
CREATE SCHEMA dw;
GO

CREATE TABLE dw.FactSolicitud (
    solicitud_key BIGINT IDENTITY(1,1) PRIMARY KEY,
    codigo_solicitud BIGINT NOT NULL,
    fecha_creacion_key INT NOT NULL,
    fecha_cierre_key INT NULL,
    agencia_key INT NOT NULL,
    problema_key INT NOT NULL,
    geografia_key INT NULL,
    estado_key INT NOT NULL,
    canal_key INT NULL,
    minutos_resolucion INT NULL,
    latitude DECIMAL(18,15) NULL,
    longitude DECIMAL(18,15) NULL
);
```

9. Proceso ETL

El proceso ETL fue implementado con SQL Server Integration Services (SSIS). El flujo extrae las solicitudes desde NYC Open Data, conserva la respuesta en RAW, convierte y valida los registros antes de pasarlos a STAGING y, finalmente, carga las dimensiones y FactSolicitud.

La separación RAW - STAGING - DW permite conservar la extracción original, aislar las reglas de calidad y mantener el modelo analítico únicamente con datos tipados. Los registros que no cumplen las validaciones se separan en salidas de rechazo y la ejecución conserva información de auditoría.

9.1 Arquitectura general

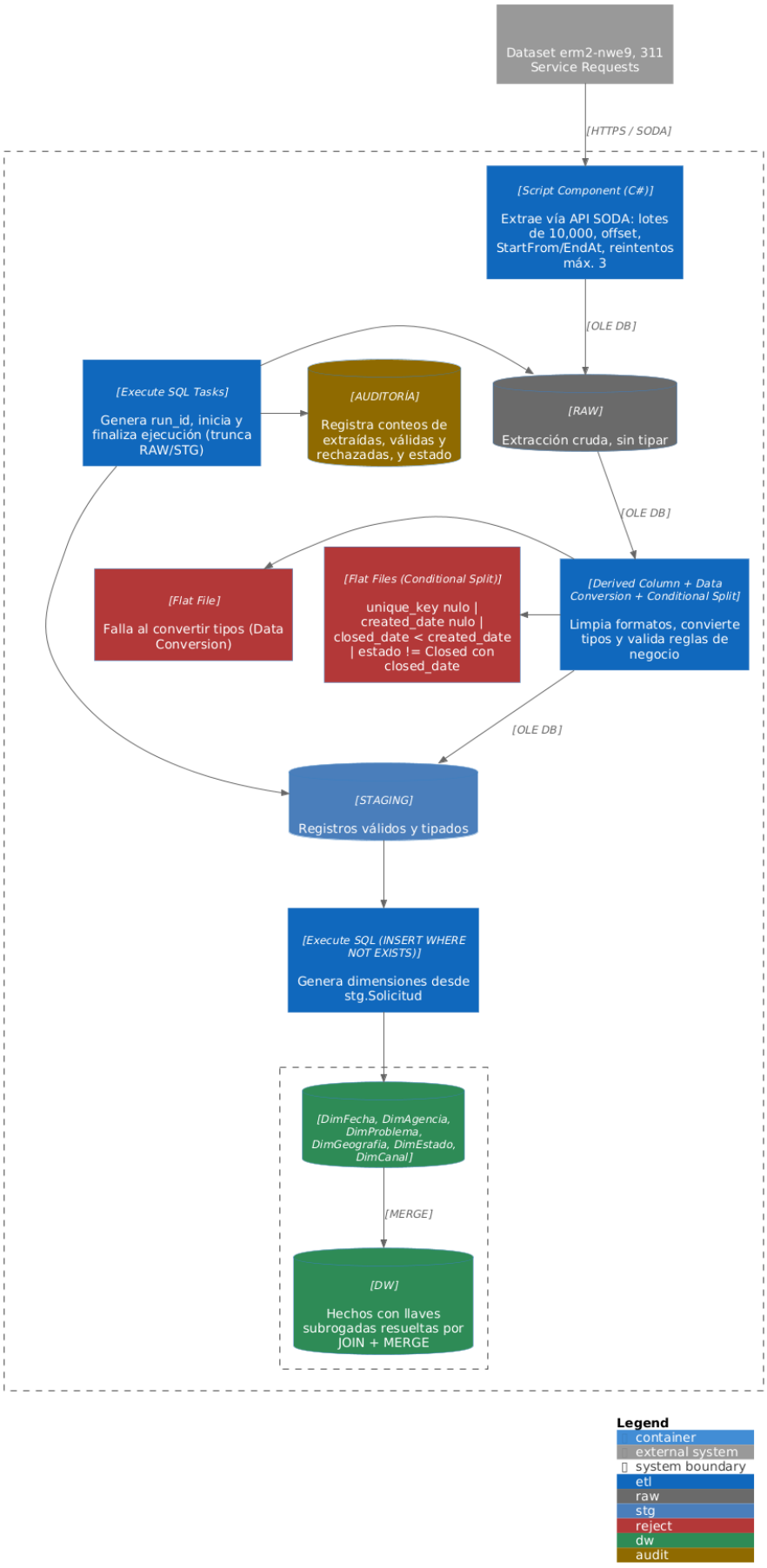


Figura 2. Arquitectura del proceso ETL para el Data Warehouse de NYC311.

9.2 Implementación del proceso ETL en SSIS

El paquete principal coordina la generación del identificador de ejecución, la preparación de RAW y STAGING, la extracción, la limpieza, la carga de dimensiones, la carga de la tabla de hechos y el cierre de la ejecución.

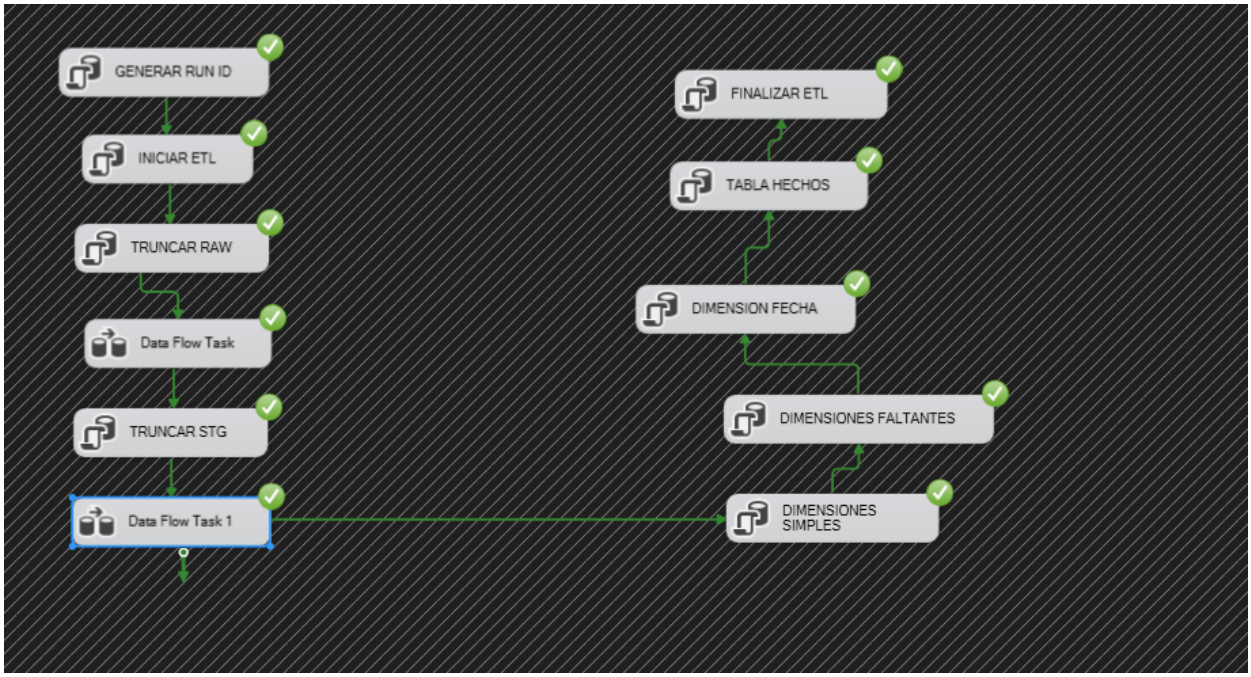


Figura 3. Flujo de control principal del paquete ETL en SSIS.

9.2.1 Extracción de datos

La extracción se realiza mediante un Script Component en C# que consume la API SODA. El componente construye la consulta a partir de StartFrom y EndAt, pagina los resultados en lotes de 10,000 registros y contempla reintentos ante fallos temporales. La salida se almacena mediante OLE DB en la capa RAW.

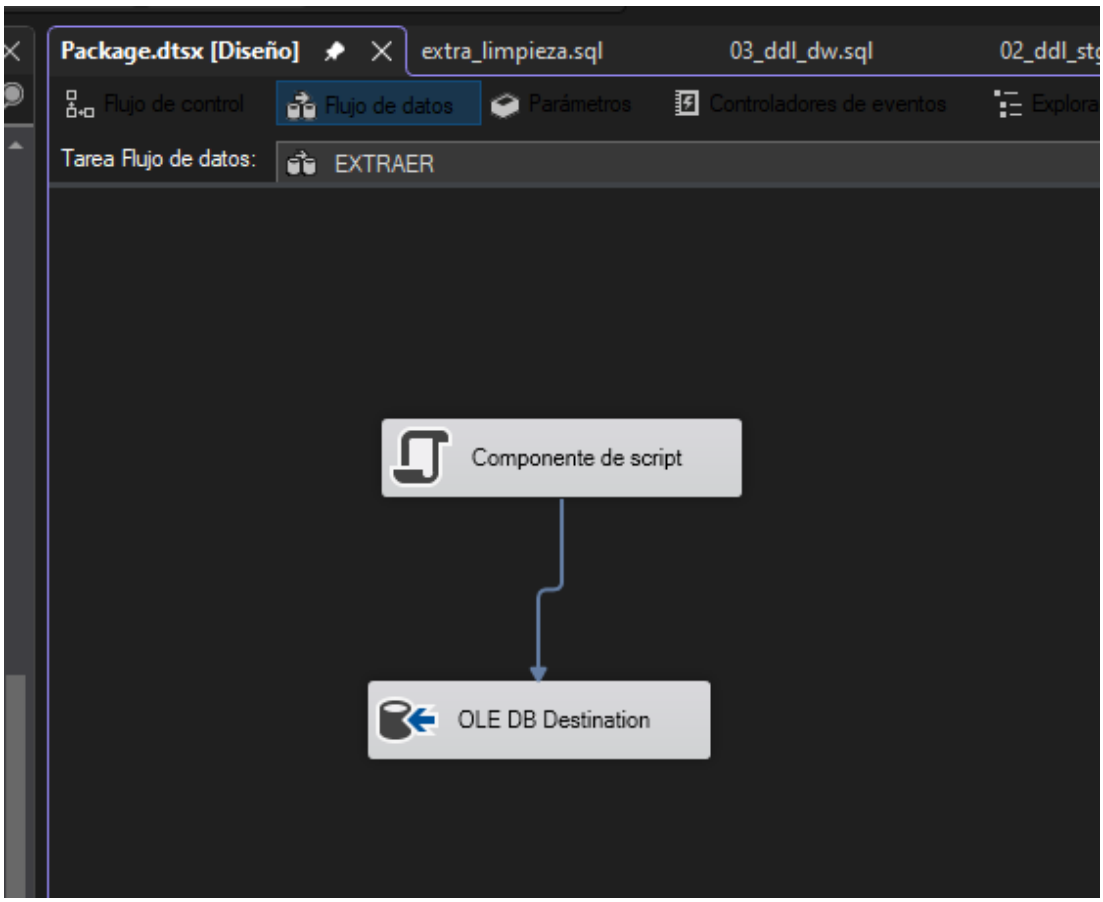


Figura 4. Flujo de extracción desde la API SODA hacia RAW.

9.2.2 Limpieza y validación de datos

Los registros de RAW son leídos mediante OLE DB y pasan por columnas derivadas y conversiones de tipo. Después, un Conditional Split evalúa las reglas de calidad. Los registros válidos continúan hacia STAGING y las inconsistencias se envían a archivos de rechazo según la causa detectada.

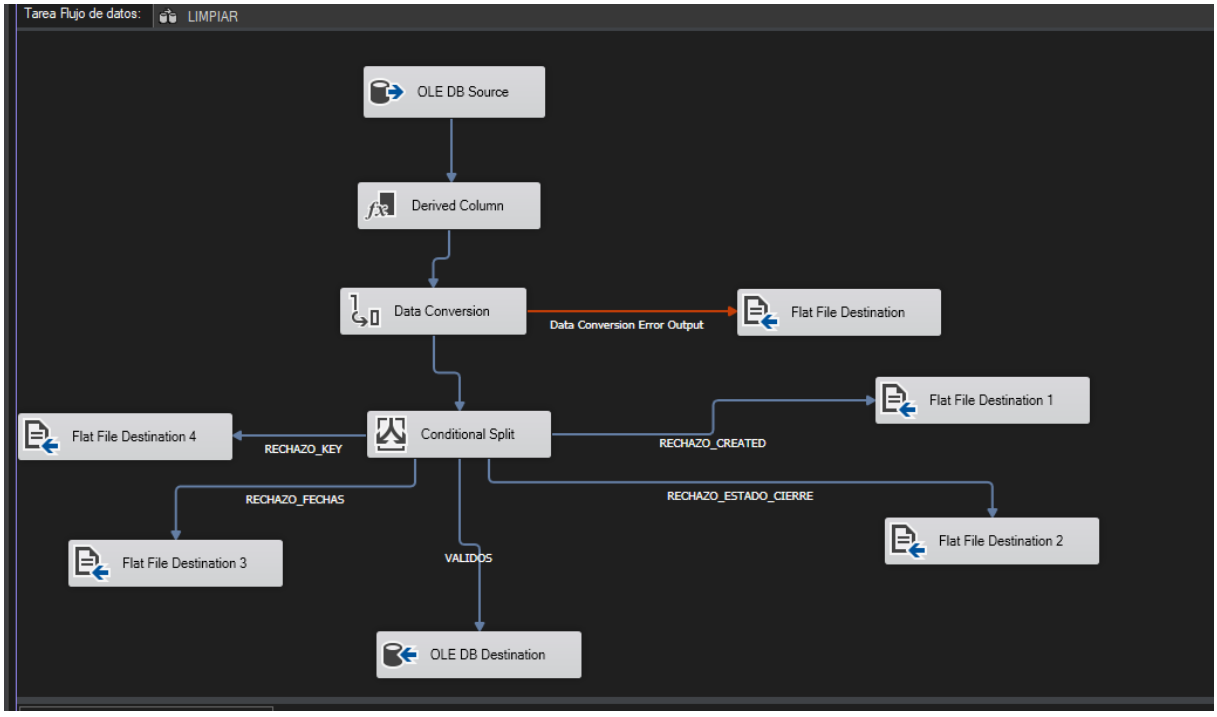


Figura 5. Flujo de limpieza, validación y clasificación de registros.

9.2.3 Reglas de calidad y salidas de rechazo

Salida	Condición controlada
VALIDOS	Registros que superan las reglas y continúan hacia STAGING.
RECHAZO_KEY	Identificador de solicitud ausente o inválido.
RECHAZO_CREATED	Fecha de creación ausente o no utilizable para el modelo.
RECHAZO_FECHAS	Inconsistencia cronológica, por ejemplo cierre anterior a la creación.
RECHAZO_ESTADO_CIERRE	Estado de la solicitud inconsistente con la información de cierre.
Error de conversión	Valor que no puede convertirse al tipo de dato esperado.

9.2.4 Carga de dimensiones y tabla de hechos

Una vez cargado STAGING, el paquete ejecuta la carga de dimensiones antes de construir la tabla de hechos. Las dimensiones incorporan los valores faltantes mediante sentencias de inserción controladas; sus claves internas se utilizan luego para resolver las relaciones de FactSolicitud.

La carga de hechos relaciona cada registro de STAGING con sus dimensiones y utiliza esas llaves subrogadas en FactSolicitud. El flujo del paquete deja esta dependencia explícita: primero se cargan las dimensiones, luego DimFecha y, al final, la tabla de hechos.

9.2.5 Manejo de errores y auditoría

Cada ejecución genera un run_id y registra el inicio y la finalización del proceso. La auditoría conserva los conteos de filas extraídas, válidas y rechazadas, además del estado y el detalle de la ejecución. Los errores de conversión y las reglas de calidad se separan del camino válido, de modo que una anomalía pueda revisarse sin incorporarse silenciosamente al Data Warehouse.



	run_id	inicio	fin	filas_extraidas	filas_validas	filas_rechazadas	filas_warning	estado	detalle
1	A0C374F8-B772-449D-B744-102A62DB9848	2026-08-09 16:26:39	NULL	0	0	0	0	RUNNING	NULL
2	6675ED33-1E22-4EDD-A942-DED1DEDB5A87	2026-08-09 16:25:18	NULL	0	0	0	0	RUNNING	NULL
3	A75C880A-E545-4AB0-BE43-E367A0B4852C	2026-08-09 16:35:54	2026-08-09 16:36:13	10909	10683	226	0	SUCCESS	ETL completado correctamente: RAW -> STG -> DW

Figura 6. Registro de procesos

10. Conclusiones de la fase

La Fase 1 deja implementada la infraestructura analítica necesaria para trabajar con las solicitudes de NYC311. El modelo estrella define una granularidad estable -una solicitud por fila-, una medida de tiempo de resolución y dimensiones que permiten analizar el hecho por tiempo, agencia, problema, geografía, estado y canal.

El paquete SSIS separa la extracción, el tipado y la carga final mediante las capas RAW, STAGING y DW. Las validaciones de fechas, identificadores y estado/cierre, junto con las salidas de rechazo y la auditoría por ejecución, permiten mantener trazabilidad sobre los registros que llegan al modelo.

Con esta base, las siguientes fases pueden trabajar sobre una estructura consistente sin repetir la preparación operativa de los datos. La minería y la visualización podrán utilizar las mismas dimensiones y métricas definidas en esta etapa.

11. Referencias

- **NYC Open Data.** 311 Service Requests from 2020 to Present (dataset erm2-nwe9). https://data.cityofnewyork.us/d/erm2-nwe9
- **Microsoft Learn.** SQL Server Integration Services (SSIS). https://learn.microsoft.com/sql/integration-services/sql-server-integration-services
- **Repositorio del proyecto.** catedra_fase1. https://github.com/pgap22/catedra_fase1

Anexo A. Script DDL completo

Script utilizado para crear la base de datos, el esquema dimensional, las dimensiones, la tabla de hechos y sus relaciones en SQL Server.

```
CREATE DATABASE DW_NYC311
GO
USE DW_NYC311;
GO
CREATE SCHEMA dw;
GO

/* ========================================================
   DIMENSIÓN: AGENCIA
   ======================================================== */
CREATE TABLE dw.DimAgencia (
    agencia_key INT IDENTITY(1,1) PRIMARY KEY,

    agency NVARCHAR(50) NOT NULL,
    agency_name NVARCHAR(250) NULL,

    CONSTRAINT UQ_DimAgencia
        UNIQUE (agency)
);
GO


/* ========================================================
   DIMENSIÓN: ESTADO
   ======================================================== */
CREATE TABLE dw.DimEstado (
    estado_key INT IDENTITY(1,1) PRIMARY KEY,

    status NVARCHAR(100) NOT NULL,

    CONSTRAINT UQ_DimEstado
        UNIQUE (status)
);
GO
/* ========================================================
   DIMENSIÓN: CANAL
   ======================================================== */
CREATE TABLE dw.DimCanal (
    canal_key INT IDENTITY(1,1) PRIMARY KEY,

    open_data_channel_type NVARCHAR(100) NOT NULL,

    CONSTRAINT UQ_DimCanal
        UNIQUE (open_data_channel_type)
);
GO


/* ========================================================
   DIMENSIÓN: PROBLEMA
   ======================================================== */
CREATE TABLE dw.DimProblema (
    problema_key INT IDENTITY(1,1) PRIMARY KEY,

    complaint_type NVARCHAR(250) NOT NULL,
    descriptor NVARCHAR(500) NULL,
    descriptor_2 NVARCHAR(500) NULL
);
GO



/* ========================================================
   DIMENSIÓN: GEOGRAFÍA
   ======================================================== */
CREATE TABLE dw.DimGeografia (
    geografia_key INT IDENTITY(1,1) PRIMARY KEY,

    borough NVARCHAR(100) NULL,
    incident_zip NVARCHAR(20) NULL,
    community_board NVARCHAR(100) NULL,
    council_district INT NULL,
```

```
    police_precinct NVARCHAR(50) NULL
);
GO


/* ========================================================
   DIMENSIÓN: FECHA
   Jerarquía OLAP:
   Año → Trimestre → Mes → Día
   ======================================================== */
CREATE TABLE dw.DimFecha (
    fecha_key INT PRIMARY KEY,       -- YYYYMMDD

    fecha DATE NOT NULL,

    anio SMALLINT NOT NULL,
    trimestre TINYINT NOT NULL,

    mes TINYINT NOT NULL,
    nombre_mes NVARCHAR(20) NOT NULL,

    dia TINYINT NOT NULL,
    dia_semana TINYINT NOT NULL,
    nombre_dia NVARCHAR(20) NOT NULL,

    CONSTRAINT UQ_DimFecha
        UNIQUE (fecha)
);
GO


/* ========================================================
   TABLA DE HECHOS: SOLICITUD
   Granularidad:
   1 fila = 1 solicitud 311
   ======================================================== */
CREATE TABLE dw.FactSolicitud (
    solicitud_key BIGINT IDENTITY(1,1) PRIMARY KEY,

    codigo_solicitud BIGINT NOT NULL,

    /* Dimensión tiempo */
    fecha_creacion_key INT NOT NULL,
    fecha_cierre_key INT NULL,

    /* Dimensiones */
    agencia_key INT NOT NULL,
    problema_key INT NOT NULL,
    geografia_key INT NULL,
    estado_key INT NOT NULL,
    canal_key INT NULL,

    /* Medida */
    minutos_resolucion INT NULL,

    /* Coordenada exacta de la solicitud */
    latitude DECIMAL(18,15) NULL,
    longitude DECIMAL(18,15) NULL,

    CONSTRAINT UQ_FactSolicitud
        UNIQUE (codigo_solicitud),

    CONSTRAINT FK_FactSolicitud_FechaCreacion
        FOREIGN KEY (fecha_creacion_key)
        REFERENCES dw.DimFecha(fecha_key),

    CONSTRAINT FK_FactSolicitud_FechaCierre
        FOREIGN KEY (fecha_cierre_key)
        REFERENCES dw.DimFecha(fecha_key),

    CONSTRAINT FK_FactSolicitud_Agencia
        FOREIGN KEY (agencia_key)
        REFERENCES dw.DimAgencia(agencia_key),

    CONSTRAINT FK_FactSolicitud_Problema
        FOREIGN KEY (problema_key)
        REFERENCES dw.DimProblema(problema_key),
```

```
    CONSTRAINT FK_FactSolicitud_Geografia
        FOREIGN KEY (geografia_key)
        REFERENCES dw.DimGeografia(geografia_key),

    CONSTRAINT FK_FactSolicitud_Estado
        FOREIGN KEY (estado_key)
        REFERENCES dw.DimEstado(estado_key),

    CONSTRAINT FK_FactSolicitud_Canal
        FOREIGN KEY (canal_key)
        REFERENCES dw.DimCanal(canal_key)
);
GO
```